COMPUTER SCIENCE ENGINEERING · CAMPUS PLACEMENT

Campus Interview Preparation Guide

360 Questions · **12** Topics · Easy / Medium / Hard

TOPICS COVERED

API · React · JavaScript · Agentic AI · Python · Express.js · Node.js · MySQL / PostgreSQL · Postman · RAG · Git & GitHub · AES / Networks

CREATED & COMPILED BY

Ramana Ganthan

linkedin.com/in/ramanaganthans

github.com/ramanaganthan7

2026

1. API (Application Programming Interface)

Subtopics: REST Principles | HTTP Methods & Status Codes | Authentication & Security

---

EASY (10 QUESTIONS)

Q1. What is an API?

> An API (Application Programming Interface) is a set of rules and protocols that allows one software application to communicate with another. It defines how requests and responses should be formatted.

Q2. What does REST stand for?

> REST stands for Representational State Transfer. It is an architectural style for designing networked applications using stateless communication over HTTP.

Q3. What are the main HTTP methods used in REST APIs?

> GET (retrieve data), POST (create data), PUT (update/replace data), PATCH (partial update), DELETE (remove data).

Q4. What is an HTTP status code 200?

> 200 OK — The request was successful and the server returned the requested data.

Q5. What is the difference between GET and POST?

> GET retrieves data and sends parameters in the URL (visible, cached, bookmarkable). POST sends data in the request body (not visible in URL, used for creating/submitting data).

Q6. What is a JSON?

> JSON (JavaScript Object Notation) is a lightweight data-interchange format that is easy for humans to read and write, and easy for machines to parse and generate. It is the most common format for API responses.

Q7. What HTTP status code means "Not Found"?

> 404 Not Found — The server cannot find the requested resource.

Q8. What is a URI vs URL?

> A URL (Uniform Resource Locator) is a specific type of URI (Uniform Resource Identifier) that provides the location of a resource along with the protocol to access it (e.g., https://api.example.com/users).

Q9. What is an API endpoint?

> An API endpoint is a specific URL where an API can be accessed. Each endpoint corresponds to a specific function or resource (e.g., /api/users, /api/products/123).

Q10. What does idempotent mean in REST?

> An operation is idempotent if calling it multiple times produces the same result. GET, PUT, DELETE are idempotent. POST is not idempotent.

MEDIUM (10 QUESTIONS)

Q11. What is the difference between PUT and PATCH?

> PUT replaces the entire resource with the provided data (full update). PATCH applies a partial update — only the provided fields are updated, the rest remain unchanged.

Q12. What are the HTTP status codes 401 vs 403?

> 401 Unauthorized — The client is not authenticated (no valid credentials). 403 Forbidden — The client is authenticated but does not have permission to access the resource.

Q13. What is CORS and why does it occur?

> CORS (Cross-Origin Resource Sharing) is a browser security policy that blocks requests from a different origin (domain/port/protocol). It is resolved by the server sending appropriate headers like `Access-Control-Allow-Origin`.

Q14. What is the difference between stateless and stateful APIs?

> Stateless: Each request contains all information needed; the server stores no session state between requests (REST principle). Stateful: The server maintains session state across requests (e.g., traditional web sessions).

Q15. What is API versioning and why is it important?

> API versioning allows multiple versions of an API to coexist (e.g., /api/v1/users, /api/v2/users). It is important so existing clients are not broken when new changes are introduced.

Q16. What is the difference between authentication and authorization?

> Authentication verifies WHO you are (e.g., login with username/password). Authorization determines WHAT you are allowed to do (e.g., admin can delete, user can only read).

Q17. What is JWT and how does it work?

> JWT (JSON Web Token) is a compact, self-contained token used for authentication. It has three parts: Header (algorithm), Payload (claims/data), and Signature. The server signs it; the client sends it in the Authorization header with each request.

Q18. What is rate limiting in APIs?

> Rate limiting restricts how many requests a client can make in a given time period to prevent abuse and ensure fair usage (e.g., 100 requests per minute per IP). Returns 429 Too Many Requests when exceeded.

Q19. What is the difference between SOAP and REST?

> REST: Lightweight, uses HTTP, supports JSON/XML, stateless, flexible. SOAP: Protocol with strict standards, XML only, has built-in error handling (WSDL), heavier, used in enterprise/financial systems.

Q20. What is an API Gateway?

> An API Gateway is a server that acts as an entry point for all API calls. It handles routing, authentication, rate limiting, load balancing, and logging — a single entry point for microservices.

HARD (10 QUESTIONS)

Q21. Explain the Richardson Maturity Model.

> It grades REST API maturity in 4 levels: Level 0 (single URI, single method — RPC style), Level 1 (multiple URIs/resources), Level 2 (HTTP verbs used correctly), Level 3 (HATEOAS — responses include links to related actions). True REST is Level 3.

Q22. What is HATEOAS?

> HATEOAS (Hypermedia As The Engine Of Application State) is a REST constraint where API responses include hyperlinks to related actions/resources, allowing clients to discover functionality dynamically without hardcoding URLs.

Q23. What is OAuth 2.0 and its grant types?

> OAuth 2.0 is an authorization framework. Grant types: Authorization Code (web apps), Implicit (deprecated, SPAs), Client Credentials (machine-to-machine), Resource Owner Password Credentials (trusted apps), Refresh Token (get new access token).

Q24. What is the difference between long polling, SSE, and WebSockets?

> Long Polling: Client repeatedly requests; server holds the response until new data. SSE (Server-Sent Events): One-way server→client stream over HTTP. WebSockets: Full-duplex, persistent, bidirectional TCP connection — best for real-time apps like chat.

Q25. How do you handle API backward compatibility?

> Strategies: versioning (/v1, /v2), making fields optional, deprecation warnings, supporting old fields alongside new ones, using feature flags, providing migration guides, and setting sunset dates for old versions.

Q26. What is GraphQL and how does it differ from REST?

> GraphQL is a query language where the client specifies exactly what data it needs in a single request. Unlike REST (multiple endpoints, fixed responses), GraphQL has one endpoint, prevents over-fetching and under-fetching, and is strongly typed with a schema.

Q27. What is idempotency key in API design?

> An idempotency key is a unique client-generated key sent in the request header to make non-idempotent operations (like POST payments) safe to retry. The server stores the result against the key and returns it if the same key is used again.

Q28. Explain the concept of API throttling vs rate limiting.

> Rate Limiting: Hard cap on number of requests in a time window (e.g., 100/min). Throttling: Slows down response rather than blocking — degrades service gracefully. Both protect the server from overload but throttling is softer.

Q29. What are the security vulnerabilities specific to APIs (OWASP API Top 10)?

> Key ones: Broken Object Level Authorization (BOLA), Broken Authentication, Excessive Data Exposure, Lack of Resources/Rate Limiting, Broken Function Level Authorization, Mass Assignment, Security Misconfiguration, Injection, Improper Asset Management, Insufficient Logging.

Q30. What is gRPC and when would you use it over REST?

> gRPC is a high-performance RPC framework using Protocol Buffers (binary serialization) over HTTP/2. Use it over REST when you need low latency, bidirectional streaming, strong typing, or high-throughput microservice-to-microservice communication.

---

2. React

Subtopics: Components & JSX | Hooks & State | Performance & Lifecycle

---

EASY (10 QUESTIONS)

Q1. What is React?

> React is an open-source JavaScript library developed by Facebook for building user interfaces, particularly single-page applications. It uses a component-based architecture and a virtual DOM.

Q2. What is JSX?

> JSX (JavaScript XML) is a syntax extension for JavaScript that lets you write HTML-like code inside JavaScript. It is transpiled to `React.createElement()` calls by Babel before running in the browser.

Q3. What is the difference between a functional and class component?

> Functional components are plain JavaScript functions that return JSX. Class components extend `React.Component` and have lifecycle methods. Since React 16.8, functional components with hooks are preferred.

Q4. What are props in React?

> Props (properties) are read-only inputs passed from a parent component to a child component. They allow data to flow down the component tree (one-way data flow).

Q5. What is state in React?

> State is a built-in object that stores data that can change over time within a component. When state changes, React re-renders the component.

Q6. What is the virtual DOM?

> The virtual DOM is a lightweight in-memory representation of the real DOM. React updates the virtual DOM first, diffs it with the previous version, and only updates the changed parts in the real DOM — making updates efficient.

Q7. What is useState hook?

> `useState` is a React hook that adds state to functional components. It returns an array: `[currentValue, setterFunction]`. Example: `const [count, setCount] = useState(0)`.

Q8. What does useEffect do?

> `useEffect` runs side effects in functional components (data fetching, subscriptions, DOM updates). It runs after render. The dependency array controls when it re-runs.

Q9. What is the key prop in React lists?

> The `key` prop is a unique identifier given to list items. It helps React efficiently identify which items changed, were added, or removed during reconciliation.

Q10. What is a React fragment?

> A React fragment (`<React.Fragment>` or `<>`) lets you group multiple elements without adding an extra DOM node.

MEDIUM (10 QUESTIONS)

Q11. Explain the React component lifecycle.

> Mounting: constructor → render → componentDidMount. Updating: render → componentDidUpdate. Unmounting: componentWillUnmount. In hooks: useEffect with [] = componentDidMount, useEffect with deps = componentDidUpdate, cleanup function = componentWillUnmount.

Q12. What is the difference between controlled and uncontrolled components?

> Controlled: form inputs whose values are controlled by React state (value + onChange). Uncontrolled: form inputs that store their own state in the DOM, accessed via refs. Controlled components are preferred for validation.

Q13. What is useCallback and useMemo?

> `useCallback` memoizes a function reference to prevent re-creation on every render. `useMemo` memoizes a computed value. Both take a dependency array and are used for performance optimization.

Q14. What is prop drilling and how do you avoid it?

> Prop drilling is passing props through multiple intermediate components that don't need them. Avoid it using Context API, Redux, Zustand, or other state management solutions.

Q15. What is React Context API?

> Context API provides a way to share values (like theme, user, language) across the component tree without prop drilling. Uses `createContext`, `Provider`, and `useContext`.

Q16. What is useRef and when would you use it?

> `useRef` returns a mutable ref object that persists across renders. Uses: accessing DOM elements directly, storing mutable values that don't trigger re-renders (like timers, previous values).

Q17. What is the difference between useEffect dependency array scenarios?

> No array: runs after every render. Empty array `[]`: runs once after mount. With deps `[a, b]`: runs when a or b changes. Cleanup function: runs before next effect or on unmount.

Q18. What is React.memo?

> `React.memo` is a HOC that memoizes a functional component, preventing re-render if props haven't changed (shallow comparison). Used for performance optimization of child components.

Q19. What is code splitting in React?

> Code splitting divides the bundle into chunks loaded on demand. React supports it via `React.lazy()` and `Suspense`, or dynamic `import()`. This reduces initial load time.

Q20. What is the difference between Redux and Context API?

> Context API: built-in, good for low-frequency updates (theme, auth). Redux: external library with actions/reducers/store, DevTools, middleware support, better for complex state with frequent updates.

HARD (10 QUESTIONS)

Q21. How does React reconciliation work?

> React uses a diffing algorithm (Fiber) to compare the new virtual DOM tree with the previous one. It reconciles by type (same component = update, different = replace), uses keys for lists, and batches DOM updates. Fiber allows interruptible rendering.

Q22. What is React Fiber architecture?

> React Fiber (React 16+) is a complete rewrite of the reconciliation engine. It enables incremental rendering — splitting render work into units, pausing/resuming work, prioritizing updates (concurrent mode), and enabling features like Suspense and transitions.

Q23. What are React Server Components?

> React Server Components (RSC) run on the server and never ship JavaScript to the client. They can directly access databases/backend, have zero bundle impact, and can be interleaved with client components. They don't have state or effects.

Q24. Explain the concept of lifting state up vs. state colocation.

> Lifting state up: moving shared state to the nearest common ancestor so multiple children can access it. State colocation: keeping state as close as possible to where it's used. The principle is to colocate first, lift only when necessary.

Q25. What is the Stale Closure problem in React hooks?

> A stale closure occurs when a useEffect or useCallback captures an old value of a variable from a previous render. Solved by including the variable in the dependency array or using useRef for mutable values that shouldn't trigger re-renders.

Q26. How does batching work in React 18?

> React 18 introduced automatic batching — multiple state updates inside event handlers, setTimeout, Promises, and native events are batched into a single re-render. Previously only updates in React event handlers were batched. Use `flushSync` to opt out.

Q27. What is the useReducer hook and when to use it over useState?

> `useReducer(reducer, initialState)` manages complex state logic with a reducer function (like Redux). Use it when state transitions depend on previous state, multiple sub-values are related, or next state depends on complex logic. Returns `[state, dispatch]`.

Q28. What are the rules of React hooks?

> 1. Only call hooks at the top level (not inside loops, conditions, or nested functions). 2. Only call hooks from React functional components or custom hooks. These rules ensure hooks are called in the same order every render.

Q29. What is concurrent rendering in React?

> Concurrent rendering allows React to prepare multiple versions of the UI simultaneously and interrupt, pause, or abandon renders. Enables `useTransition` (mark updates as non-urgent), `useDeferredValue`, and Suspense for data fetching.

Q30. What is hydration in React (SSR)?

> Hydration is the process where React takes server-rendered HTML and attaches event handlers/interactivity to make it a fully interactive React app on the client. React "hydrates" the static HTML without re-rendering it from scratch.

---

3. JavaScript

Subtopics: Core Concepts & Types | Async & Closures | ES6+ & DOM

EASY (10 QUESTIONS)

Q1. What are the data types in JavaScript?

> Primitive: String, Number, BigInt, Boolean, Undefined, Null, Symbol. Reference: Object (includes Arrays, Functions, Dates). `typeof null` returns "object" (historical bug).

Q2. What is the difference between == and ===?

> `==` is loose equality — performs type coercion before comparison. `===` is strict equality — checks both value and type without coercion. Always prefer `===`.

Q3. What is var, let, and const?

> `var`: function-scoped, hoisted, can be re-declared. `let`: block-scoped, hoisted but not initialized (TDZ), cannot be re-declared. `const`: block-scoped, must be initialized, cannot be reassigned (but object properties can change).

Q4. What is hoisting?

> Hoisting is JavaScript's behavior of moving declarations to the top of their scope before code execution. `var` declarations are hoisted and initialized as `undefined`. `let`/`const` are hoisted but remain in the Temporal Dead Zone until declaration.

Q5. What is a closure?

> A closure is a function that retains access to its outer (enclosing) scope even after the outer function has returned. It "closes over" variables from the parent scope.

Q6. What is the difference between null and undefined?

> `undefined`: variable declared but not assigned a value (or function returns nothing). `null`: intentional absence of value, explicitly set by the developer. `typeof undefined` is "undefined"; `typeof null` is "object".

Q7. What is an arrow function?

> Arrow functions (`() => {}`) are a concise syntax for functions. Key differences: they do not have their own `this`, `arguments`, or `prototype`. They lexically bind `this` from the surrounding context.

Q8. What is a Promise?

> A Promise is an object representing the eventual completion or failure of an asynchronous operation. States: Pending, Fulfilled, Rejected. Uses `.then()` for success, `.catch()` for errors.

Q9. What is event bubbling?

> Event bubbling is when an event triggered on a child element propagates up through its parent elements. You can stop it with `event.stopPropagation()`.

Q10. What is the difference between forEach and map?

> `forEach` iterates over an array and executes a callback but returns `undefined`. `map` creates and returns a new array with the results of calling the callback on each element.

MEDIUM (10 QUESTIONS)

Q11. What is the event loop in JavaScript?

> JavaScript is single-threaded. The event loop monitors the call stack and callback queue. When the call stack is empty, it moves tasks from the queue to the stack. Microtasks (Promises) are processed before macrotasks (setTimeout).

Q12. What is the difference between call, apply, and bind?

> All three set `this` explicitly. `call(thisArg, arg1, arg2)`: invokes immediately with individual args. `apply(thisArg, [args])`: invokes immediately with args as array. `bind(thisArg, arg1)`: returns a new function with `this` bound, does not invoke immediately.

Q13. What is prototype chain in JavaScript?

> Every JavaScript object has a `__proto__` property pointing to its prototype. When accessing a property, JS looks up the prototype chain until it finds the property or reaches `null` (Object.prototype). This enables inheritance.

Q14. Explain async/await.

> `async/await` is syntactic sugar over Promises. An `async` function always returns a Promise. `await` pauses execution inside the async function until the Promise resolves. Makes async code look synchronous. Error handling uses try/catch.

Q15. What is destructuring in JavaScript?

> Destructuring extracts values from arrays or objects into distinct variables. Array: `const [a, b] = [1, 2]`. Object: `const { name, age } = person`. Supports default values and renaming.

Q16. What is the spread operator vs rest parameter?

> Spread (`...arr`): expands an iterable into individual elements (e.g., merging arrays/objects, passing args). Rest (`...args`): collects multiple function arguments into an array. Same syntax, opposite behavior.

Q17. What are WeakMap and WeakSet?

> WeakMap/WeakSet hold weak references to objects — they don't prevent garbage collection if there are no other references. Keys must be objects. Not iterable. Used for private data or caching without memory leaks.

Q18. What is the difference between synchronous and asynchronous code in JS?

> Synchronous: code executes line by line, blocking until each operation completes. Asynchronous: operations (I/O, timers, fetch) are offloaded and the rest of the code continues. Callbacks, Promises, async/await handle async results.

Q19. What is optional chaining (?.) and nullish coalescing (??) in ES2020?

> Optional chaining: `obj?.prop?.subProp` — returns `undefined` instead of throwing if a property is null/undefined. Nullish coalescing: `a ?? b` — returns `b` only if `a` is null or undefined (unlike `||` which triggers on any falsy value).

Q20. What is a generator function?

> A generator function (`function*`) can pause and resume execution using `yield`. It returns an iterator object. Each call to `.next()` runs until the next `yield`. Used for lazy evaluation, infinite sequences, and async flows.

HARD (10 QUESTIONS)

Q21. Explain the difference between microtasks and macrotasks.

> Macrotasks (task queue): setTimeout, setInterval, setImmediate, I/O. Microtasks (microtask queue): Promise.then/catch/finally, queueMicrotask, MutationObserver. After each macrotask, ALL microtasks are processed before the next macrotask.

Q22. What is memoization in JavaScript?

> Memoization caches the results of function calls based on input arguments. Subsequent calls with the same arguments return the cached result. Implemented by storing results in a Map/object. Optimizes pure functions with expensive computation.

Q23. What is the Temporal Dead Zone (TDZ)?

> The TDZ is the period between entering a scope and the variable declaration being initialized. `let` and `const` variables exist in TDZ from block start until their declaration line. Accessing them throws `ReferenceError`.

Q24. How does garbage collection work in JavaScript?

> JS uses mark-and-sweep: the GC marks all objects reachable from roots (global, stack), then sweeps unmarked objects. Modern V8 uses generational GC (young/old generations), incremental marking, and concurrent GC to minimize pauses.

Q25. What is currying in JavaScript?

> Currying transforms a function with multiple arguments into a sequence of functions, each taking one argument. Example: `f(a, b, c)` becomes `f(a)(b)(c)`. Enables partial application and function composition.

Q26. Explain the module system in JavaScript (CommonJS vs ES Modules).

> CommonJS (Node.js): `require()` / `module.exports` — synchronous, dynamic loading. ES Modules (ESM): `import` / `export` — static, tree-shakeable, asynchronous loading, uses strict mode by default. Browsers support ESM natively. Node.js supports both.

Q27. What is the difference between deep copy and shallow copy?

> Shallow copy copies only the first level; nested objects are still references (e.g., Object.assign, spread). Deep copy duplicates all levels recursively (JSON.parse(JSON.stringify(obj)) — loses functions/undefined; use structuredClone() in modern JS).

Q28. What is Proxy in JavaScript?

> `Proxy` wraps an object and intercepts operations (get, set, delete, etc.) via handler "traps". Used for validation, logging, reactive data binding (like Vue 3's reactivity), and meta-programming.

Q29. How does the V8 engine optimize JavaScript code?

> V8 uses JIT (Just-In-Time) compilation: interprets bytecode first via Ignition, then identifies "hot" functions and compiles them to optimized machine code via TurboFan. Deoptimizes if type assumptions break. Uses hidden classes for object property access optimization.

Q30. What are the memory leak patterns in JavaScript?

> Common patterns: global variables, forgotten timers/intervals, detached DOM references, closures holding references, event listeners not removed. Detection: Chrome DevTools heap snapshots, performance tab memory timeline.

---

4. Agentic AI

Subtopics: LLM Agents & Tools | Multi-Agent Systems | Agent Frameworks

---

EASY (10 QUESTIONS)

Q1. What is an AI agent?

> An AI agent is an LLM-powered system that can perceive its environment, reason about a task, take actions (via tools/functions), and iterate until a goal is achieved — going beyond simple question-answering.

Q2. What is the ReAct pattern in AI agents?

> ReAct (Reasoning + Acting) is a prompting framework where the agent alternates between Thought (reasoning about what to do), Action (calling a tool), and Observation (processing the result), iterating until the task is complete.

Q3. What are tools in the context of AI agents?

> Tools are functions or APIs that an AI agent can call to perform actions: web search, code execution, file read/write, database queries, API calls. They extend the agent's capabilities beyond just generating text.

Q4. What is an AI agent loop?

> The agent loop is: receive task → reason/plan → select and call a tool → observe result → decide next action → repeat until task is complete or a stopping condition is met.

Q5. What is the difference between a single-agent and multi-agent system?

> Single-agent: one LLM handles the full task. Multi-agent: multiple specialized agents work together — orchestrator assigns subtasks to sub-agents, enabling parallelism and specialization.

Q6. What is function calling in LLMs?

> Function calling allows LLMs (like GPT-4, Claude) to output structured JSON specifying a function name and arguments instead of plain text. The application executes the function and returns the result to the model.

Q7. What is an orchestrator agent?

> An orchestrator agent coordinates other agents or tools — it breaks down a high-level task, delegates subtasks to specialized sub-agents, and synthesizes their results into a final answer.

Q8. What is AutoGen?

> AutoGen is a Microsoft framework for building multi-agent systems where multiple AI agents (with different roles) can converse, collaborate, and execute code to solve tasks.

Q9. What is LangChain?

> LangChain is a framework for building LLM-powered applications. It provides abstractions for chains, agents, tools, memory, and retrieval — making it easier to build complex AI workflows.

Q10. What is memory in an AI agent?

> Agent memory stores context across interactions: short-term memory (current conversation context window), long-term memory (vector database for past interactions), and episodic memory (specific past experiences).

MEDIUM (10 QUESTIONS)

Q11. What is the difference between chains and agents in LangChain?

> Chains: fixed, predetermined sequence of steps. Agents: dynamic — the LLM decides at each step which tool to call based on the task and previous observations. Agents are more flexible but less predictable.

Q12. What is a system prompt in agentic AI?

> A system prompt is an instruction given to the LLM at the start defining its role, capabilities, constraints, available tools, and behavior. It shapes how the agent interprets and responds to all subsequent messages.

Q13. What is tool/function schema in agent systems?

> A tool schema is a structured description of a tool (name, description, parameters with types/descriptions) provided to the LLM so it knows when and how to call it. The LLM uses this schema to decide which tool and what arguments to use.

Q14. How do you handle agent errors and retries?

> Strategies: try/except around tool calls, retry with exponential backoff, return error message to LLM as an observation and let it try a different approach, set max_iterations to prevent infinite loops, fallback to human-in-the-loop.

Q15. What is the difference between zero-shot and few-shot prompting in agents?

> Zero-shot: agent uses only the system prompt and task description, no examples. Few-shot: provide example inputs/outputs/tool-calls in the prompt to guide behavior. Few-shot improves reliability for complex or unusual tasks.

Q16. What is agent memory using vector databases?

> Long-term agent memory stores past interactions as vector embeddings in a vector database. When a new task arrives, semantically similar past memories are retrieved and injected into the context, enabling the agent to "remember" across sessions.

Q17. What is the difference between OpenAI Assistants API and custom agent implementations?

> OpenAI Assistants API: managed, built-in threads/memory/tool-calling, file retrieval, code interpreter. Custom agents (LangChain, AutoGen): more control, framework-specific, portable across models. Assistants API is simpler but less flexible.

Q18. What is chain-of-thought (CoT) prompting?

> CoT prompting encourages the model to reason step-by-step before giving a final answer (e.g., "Let's think step by step"). It improves performance on reasoning, math, and multi-step tasks by externalizing the reasoning process.

Q19. What is a planner agent?

> A planner agent decomposes a complex task into a sequence of subtasks/steps before execution. Examples: Plan-and-Execute (LangChain), OpenAI task planning. Separates planning from execution for more reliable multi-step workflows.

Q20. What is grounding in AI agents?

> Grounding connects the agent's outputs to real-world, factual, up-to-date data — via tool calls (web search, database), RAG, or API calls. It reduces hallucinations and ensures responses are based on actual data.

HARD (10 QUESTIONS)

Q21. What is the Agent SDK (Claude/Anthropic)?

> The Anthropic Agent SDK enables building multi-agent applications with Claude. It provides abstractions for creating agents with tools, managing context, spawning sub-agents, and orchestrating workflows with Claude as the underlying model.

Q22. What are the key challenges in building reliable AI agents?

> Hallucination in tool selection, infinite loops, context window limits, error cascading in multi-agent systems, inconsistent JSON output, latency, cost, security (prompt injection), and difficulty in evaluating agent correctness.

Q23. What is prompt injection in agentic AI systems?

> Prompt injection is when malicious content in tool outputs or retrieved documents contains instructions that hijack the agent's behavior. Mitigations: separate system/user/tool-output contexts, input sanitization, trust boundaries, human-in-the-loop.

Q24. Explain the difference between reactive agents and deliberative agents.

> Reactive agents: respond to immediate stimuli without planning (ReAct pattern). Deliberative agents: build internal models, plan ahead before acting (BDI — Belief-Desire-Intention model). Modern LLM agents blend both.

Q25. What is the context window management challenge in long-running agents?

> Long-running agents can exceed the LLM's context window. Strategies: summarizing past steps, compressing tool outputs, storing detailed history in external memory (vector DB), sliding window approaches, and using models with larger context windows.

Q26. How do you evaluate AI agent performance?

> Metrics: task completion rate, step efficiency (fewer steps = better), tool call accuracy, hallucination rate, latency, cost per task. Evaluation frameworks: AgentBench, GAIA, custom task suites with ground truth. Human evaluation for open-ended tasks.

Q27. What is self-reflection/self-critique in AI agents?

> Self-reflection is when an agent evaluates its own output before returning it, identifying errors and improving. Implementations: Reflexion (agent generates feedback on failures and retries), Self-RAG (agent decides when to retrieve), Constitutional AI.

Q28. What is the role of structured outputs in agentic systems?

> Structured outputs (JSON mode, constrained generation) ensure the LLM produces valid, parseable responses. This is critical for tool calling reliability, data extraction, and workflow orchestration where downstream code expects specific formats.

Q29. What is the difference between LangGraph and LangChain?

> LangChain provides linear chains and simple agents. LangGraph extends it with a graph-based approach — nodes are states/actions, edges are transitions, supports cycles (for loops/retries), conditional branching, and complex stateful multi-agent workflows.

Q30. How do multi-agent systems handle task delegation and result aggregation?

> Orchestrator decomposes task, assigns to specialized agents via tool calls or messaging. Sub-agents execute and return results. Orchestrator aggregates using strategies: sequential (chain), parallel (fan-out/fan-in), hierarchical. Communication via shared memory, message passing, or direct function calls.

---

5. Python

Subtopics: Core Python & OOP | Data Structures & Algorithms | Libraries & Advanced

---

EASY (10 QUESTIONS)

Q1. What is Python and what are its key features?

> Python is a high-level, interpreted, dynamically-typed, general-purpose programming language. Key features: readable syntax, dynamic typing, garbage collection, extensive standard library, multi-paradigm (OOP, functional, procedural).

Q2. What is the difference between a list and a tuple in Python?

> List: mutable (can change elements), uses `[]`. Tuple: immutable (cannot change after creation), uses `()`. Tuples are faster and can be used as dictionary keys. Use tuples for fixed data.

Q3. What are Python decorators?

> A decorator is a function that takes another function and extends its behavior without modifying it. Applied with `@decorator_name`. Used for logging, authentication, caching, timing. Built-in: `@staticmethod`, `@classmethod`, `@property`.

Q4. What is the difference between `is` and `==` in Python?

> `==` checks value equality. `is` checks identity (same object in memory). Example: `[1,2] == [1,2]` is True, but `[1,2] is [1,2]` is False (different objects). Use `is` only for None checks (`if x is None`).

Q5. What is a list comprehension?

> A concise way to create lists: `[expression for item in iterable if condition]`. Example: `[x**2 for x in range(10) if x % 2 == 0]`. More readable and often faster than equivalent for loops.

**Q6. What is *args and **kwargs?

> `*args` collects extra positional arguments into a tuple. `**kwargs` collects extra keyword arguments into a dictionary. Used to write flexible functions that accept variable numbers of arguments.

Q7. What is the GIL in Python?

> The GIL (Global Interpreter Lock) is a mutex in CPython that ensures only one thread executes Python bytecode at a time. It prevents true CPU parallelism in threads but doesn't affect multiprocessing or I/O-bound threading.

Q8. What is the difference between deep copy and shallow copy in Python?

> Shallow copy (`copy.copy()`): copies the outer object but nested objects are still references. Deep copy (`copy.deepcopy()`): recursively copies all objects. Matters for nested mutable structures.

Q9. What are Python generators?

> Generators are functions that use `yield` to produce values lazily one at a time. They don't store all values in memory. Created with `yield` or generator expressions `(x for x in range(1000))`. Ideal for large datasets.

Q10. What is the difference between `__str__` and `__repr__`?

> `__repr__`: unambiguous string representation for developers (used in REPL, logs). `__str__`: readable string for end users (used by print). If `__str__` is not defined, `__repr__` is used.

MEDIUM (10 QUESTIONS)

Q11. Explain Python's MRO (Method Resolution Order).

> MRO determines the order in which base classes are searched for method resolution in multiple inheritance. Python uses C3 linearization algorithm. View with `ClassName.__mro__`. The order is: current class → left parent → right parent → object.

Q12. What are context managers and the `with` statement?

> Context managers manage resources by defining `__enter__` (setup) and `__exit__` (teardown/cleanup). The `with` statement ensures cleanup even if exceptions occur. Created via class or `@contextmanager` decorator from `contextlib`.

Q13. What is the difference between multiprocessing and multithreading in Python?

> Threading: multiple threads in same process, share memory, limited by GIL (good for I/O-bound). Multiprocessing: multiple processes with separate memory, bypasses GIL (good for CPU-bound). Use `concurrent.futures` for high-level abstraction.

Q14. What are Python's built-in data structures and their time complexities?

> List: O(1) append/access, O(n) insert/search. Dict/Set: O(1) average insert/search (hash table). Tuple: O(1) access. deque: O(1) append/pop both ends. heapq: O(log n) push/pop.

Q15. What is metaclass in Python?

> A metaclass is a class whose instances are classes. It controls class creation. `type` is the default metaclass. Custom metaclasses override `__new__` or `__init__` to modify class creation — used in ORMs (Django), API frameworks.

Q16. What is the difference between classmethod, staticmethod, and instance method?

> Instance method: takes `self` (instance), can access instance and class. `@classmethod`: takes `cls` (class), can access/modify class state, called on class. `@staticmethod`: takes no implicit first argument, just a utility function in the class namespace.

Q17. Explain asyncio in Python.

> `asyncio` is Python's async I/O framework using coroutines (`async/await`). Single-threaded concurrency — event loop switches between coroutines when one awaits an I/O operation. Better than threads for high-concurrency I/O-bound work (web servers, clients).

Q18. What is pickling in Python?

> Pickling is serializing a Python object to bytes (`pickle.dumps()`). Unpickling is deserializing back (`pickle.loads()`). Used for saving model state (ML), caching, IPC. Warning: never unpickle untrusted data (security risk).

Q19. What is the difference between `__new__` and `__init__`?

> `__new__` creates the instance (allocates memory), called before `__init__`. `__init__` initializes the instance attributes after creation. Override `__new__` for singletons, immutable types like `tuple`, or custom metaclasses.

Q20. What is monkey patching?

> Monkey patching is dynamically modifying a module, class, or function at runtime. Used in testing (mocking), quick fixes, or extending third-party code. Generally discouraged in production as it is hard to debug and maintain.

HARD (10 QUESTIONS)

Q21. How does Python's memory management work?

> CPython uses reference counting (each object tracks references; freed when count reaches 0) + cyclic garbage collector (detects reference cycles). Memory is managed in arenas → pools → blocks. Small objects (<256 bytes) use the pymalloc allocator.

Q22. What is a descriptor in Python?

> A descriptor is an object that defines `__get__`, `__set__`, or `__delete__` methods. When accessed as a class attribute, these methods intercept the operation. Used by `property`, `classmethod`, `staticmethod`. Powers Python's attribute access system.

Q23. Explain the difference between `__slots__` and regular attributes.

> `__slots__` restricts instance attributes to a fixed set, storing them in a compact array instead of `__dict__`. Benefits: reduced memory (no `__dict__`), faster attribute access. Downside: no dynamic attributes, more complex inheritance.

Q24. What is the difference between `iter()` and `next()` in Python's iterator protocol?

> `__iter__()` returns the iterator object itself. `__next__()` returns the next value, raises `StopIteration` when exhausted. Any object implementing both is an iterator. `for` loops call these implicitly.

Q25. How do you implement a thread-safe singleton in Python?

> Use a lock: define class-level `_instance = None` and `_lock = threading.Lock()`. In `__new__`, acquire lock, check if instance exists, create if not, release lock. The double-checked locking pattern minimizes lock contention.

Q26. What is the `__call__` method?

> `__call__` makes an instance callable like a function. When you call `obj()`, Python invokes `obj.__call__()`. Used in decorators implemented as classes, callable objects, and function-like interfaces.

Q27. Explain Python's import system and circular imports.

> When Python imports a module, it runs it top-to-bottom and caches it in `sys.modules`. Circular imports occur when A imports B which imports A. Solutions: move import inside function, restructure code, use `importlib`, or import the module not the object.

Q28. What is `functools.lru_cache` and how does it work internally?

> `@lru_cache(maxsize=128)` memoizes function results using a dictionary keyed by arguments. LRU (Least Recently Used) evicts the oldest unused entry when max size is reached. Arguments must be hashable. Thread-safe in Python 3.2+.

Q29. What are abstract base classes (ABCs) in Python?

> ABCs define interfaces that subclasses must implement. From `abc` module: use `@abstractmethod` to declare required methods. Instantiating a class without implementing abstract methods raises `TypeError`. Enables duck typing with explicit contracts.

Q30. How does Python's `typing` module work and what is Protocol?

> The `typing` module provides type hints for static analysis. `Protocol` (PEP 544) enables structural subtyping — any class implementing the required methods is compatible, without explicit inheritance. Checked by mypy/pyright, not at runtime by default.

---

6. Express.js

Subtopics: Routing & Middleware | Request/Response | Error Handling & Security

---

EASY (10 QUESTIONS)

Q1. What is Express.js?

Express.js is a minimal, unopinionated web application framework for Node.js. It provides routing, middleware support, and utilities to build web servers and REST APIs quickly.

Q2. How do you create a basic Express server?

`const express = require('express'); const app = express(); app.get('/', (req, res) => res.send('Hello')); app.listen(3000);`

Q3. What is middleware in Express?

Middleware are functions with access to `req`, `res`, and `next`. They execute in order and can modify request/response, end the cycle, or call `next()` to pass control to the next middleware.

Q4. What is the difference between app.use() and app.get()?

`app.use()` matches all HTTP methods and can match partial paths (used for middleware). `app.get()` only matches GET requests to the exact path (used for route handlers).

Q5. How do you read URL parameters in Express?

Route params: `app.get('/user/:id', (req, res) => res.send(req.params.id))`. Query params: `req.query.name` (from `/user?name=John`). Request body: `req.body` (requires body-parser/express.json() middleware).

Q6. What does next() do in Express middleware?

Calling `next()` passes control to the next middleware function. Not calling `next()` ends the request-response cycle. Calling `next(err)` passes control to the error-handling middleware.

Q7. How do you serve static files in Express?

> Use built-in middleware: `app.use(express.static('public'))`. This serves files from the `public` directory.

Q8. What is express.json() middleware?

> `express.json()` parses incoming requests with JSON payloads and populates `req.body`. Must be added before routes that read `req.body`.

Q9. How do you set a response status code in Express?

> `res.status(404).send('Not Found')` or `res.status(201).json({ message: 'Created' })`.

Q10. What is an Express Router?

> `express.Router()` creates modular route handlers that can be mounted at a path. Organizes routes into separate files: `const router = express.Router(); router.get('/users', handler); app.use('/api', router)`.

MEDIUM (10 QUESTIONS)

Q11. What is the order of middleware execution in Express?

> Middleware executes in the order it is defined with `app.use()`. The request passes through each middleware in sequence until `next()` is not called or a response is sent.

Q12. How do you implement error handling middleware in Express?

> Error handling middleware has 4 parameters `(err, req, res, next)`. Place it after all routes: `app.use((err, req, res, next) => { res.status(500).json({ error: err.message }) })`. Called when `next(err)` is invoked.

Q13. What is CORS middleware and how do you enable it in Express?

> Install `cors` package: `const cors = require('cors'); app.use(cors())` — allows all origins. For specific origins: `app.use(cors({ origin: 'http://frontend.com' }))`. Adds appropriate Access-Control headers.

Q14. How do you implement authentication middleware in Express?

Create middleware that checks `req.headers.authorization`, validates JWT, and sets `req.user`. If invalid, return 401. If valid, call `next()`. Apply to protected routes: `app.get('/protected', authMiddleware, handler)`.

Q15. What is the difference between `res.send()`, `res.json()`, and `res.end()`?

`res.send()`: sends string/buffer/object, sets Content-Type automatically. `res.json()`: sends JSON, sets Content-Type to application/json, stringifies objects. `res.end()`: low-level, sends no body or raw data without headers.

Q16. How do you handle async errors in Express?

Express doesn't catch async errors by default. Use try/catch with next(err): `async (req, res, next) => { try { await someAsyncOp(); } catch(err) { next(err); } }`. Or use express-async-errors package to auto-forward async rejections.

Q17. What is rate limiting in Express?

Use `express-rate-limit` package: `const limiter = rateLimit({ windowMs: 15*60*1000, max: 100 }); app.use(limiter)`. Limits each IP to 100 requests per 15 minutes, returning 429 when exceeded.

Q18. How do you implement request validation in Express?

Use `express-validator` or `joi` / `zod`: define validation rules for `req.body` / `req.params`, run validation, check results, and return 400 with errors if validation fails before processing.

Q19. What is `app.locals` vs `res.locals`?

`app.locals`: application-level variables available throughout the app's lifetime. `res.locals`: request-scoped variables available only for the current request-response cycle (e.g., set in middleware, used in template).

Q20. How do you implement file uploads in Express?

Use `multer` middleware: `const upload = multer({ dest: 'uploads/' }); app.post('/upload', upload.single('file'), (req, res) => res.json({ filename: req.file.filename }))`.

HARD (10 QUESTIONS)

Q21. How do you structure a large Express application?

> Use MVC pattern: separate routes (`/routes`), controllers (`/controllers`), services (`/services`), models (`/models`). Use Router for modularity, middleware folder for cross-cutting concerns. Apply dependency injection for testability.

Q22. How does Express handle concurrent requests?

> Node.js/Express is single-threaded but non-blocking — handles concurrency via the event loop. I/O operations are offloaded asynchronously. For CPU-intensive tasks, use worker threads or cluster module to spawn multiple instances.

Q23. What is the difference between app-level, router-level, and error-handling middleware?

> App-level: `app.use()` applies to all routes. Router-level: `router.use()` applies only to routes in that router instance. Error-handling: 4-parameter middleware `(err, req, res, next)` only invoked via `next(err)`.

Q24. How do you implement request logging in Express?

> Use `morgan` package: `app.use(morgan('combined'))` for Apache-style logs. For custom: create middleware logging `req.method`, `req.url`, `res.statusCode`, response time. Use winston/pino for structured JSON logging in production.

Q25. What security headers should you set in an Express app?

> Use `helmet` package: sets Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, X-XSS-Protection, HSTS, Referrer-Policy. `app.use(helmet())` applies all defaults. Essential for production.

Q26. How do you implement session management in Express?

> Use `express-session`: stores session ID in cookie, session data on server (memory/Redis/DB). Set `secret`, `resave: false`, `saveUninitialized: false`. Use Redis store for production to handle multiple instances.

Q27. How do you implement graceful shutdown in Express?

> Listen for `SIGTERM` / `SIGINT` signals. Call `server.close()` to stop accepting new connections. Wait for active requests to complete. Close DB connections, flush logs. Set a timeout for forced shutdown if requests take too long.

Q28. What is the difference between `express.urlencoded()` and `express.json()`?

> `express.json()`: parses `Content-Type: application/json` body. `express.urlencoded({ extended: true })`: parses `application/x-www-form-urlencoded` body (HTML form submissions). Both populate `req.body`. Need both for full form + API support.

Q29. How would you implement a circuit breaker pattern in Express?

> Use `opossum` library or custom implementation: track failure count for external service calls. When failures exceed threshold, "open" the circuit (return error immediately without calling service). After timeout, allow test request ("half-open"). Reset on success.

Q30. How do you implement API versioning in Express?

> Options: URL path versioning (`app.use('/api/v1', v1Router)`), header versioning (check `req.headers['api-version']`), query param versioning (`req.query.version`). URL path is most common and explicit. Mount different router versions at different paths.

---

7. Node.js

Subtopics: Event Loop & Architecture | Modules & npm | Streams & Performance

---

EASY (10 QUESTIONS)

Q1. What is Node.js?

> Node.js is a JavaScript runtime built on Chrome's V8 engine. It allows running JavaScript on the server side. It uses a non-blocking, event-driven I/O model, making it efficient for scalable network applications.

Q2. What is the event loop in Node.js?

> The event loop allows Node.js to perform non-blocking I/O despite being single-threaded. It offloads I/O to the OS/libuv thread pool and processes callbacks when operations complete, using phases: timers, I/O callbacks, idle/prepare, poll, check, close.

Q3. What is npm?

> npm (Node Package Manager) is the default package manager for Node.js. It manages project dependencies, scripts, and publishing packages. `package.json` defines the project; `node_modules` stores packages.

Q4. What is the difference between require and import in Node.js?

> `require` is CommonJS (synchronous, Node default). `import` is ES Modules (asynchronous, static, tree-shakeable). Use `"type": "module"` in package.json for ESM. CommonJS uses `.js` / `.cjs`, ESM uses `.mjs` or `.js` with module type.

Q5. What is package.json?

> `package.json` is the configuration file for a Node.js project. It contains: name, version, description, main entry point, scripts (npm run), dependencies, devDependencies, and other metadata.

Q6. What is the difference between dependencies and devDependencies?

> `dependencies`: packages required in production (runtime). `devDependencies`: packages only needed during development (testing, linting, building). Install dev deps with `npm install --save-dev`.

Q7. What is __dirname in Node.js?

> `__dirname` is a global variable that contains the absolute path of the directory containing the current module. Used to construct file paths relative to the current file.

Q8. What is the difference between synchronous and asynchronous file system operations in Node.js?

> `fs.readFile()` is async (non-blocking, callback). `fs.readFileSync()` is sync (blocking). Always use async in servers to avoid blocking the event loop.

Q9. What is the process object in Node.js?

> `process` is a global object providing info and control over the Node.js process: `process.env` (environment variables), `process.argv` (CLI args), `process.exit()`, `process.cwd()`, `process.pid`.

Q10. What is the purpose of .env files?

> `.env` files store environment-specific configuration as key-value pairs (API keys, DB URLs, ports). Use `dotenv` package to load them: `require('dotenv').config()`. Never commit `.env` to version control.

MEDIUM (10 QUESTIONS)

Q11. Explain the phases of the Node.js event loop.

> 1. Timers: execute `setTimeout` / `setInterval` callbacks. 2. Pending I/O callbacks. 3. Idle/Prepare. 4. Poll: retrieve new I/O events. 5. Check: `setImmediate` callbacks. 6. Close callbacks. Between phases: process.nextTick and Promise microtasks run.

Q12. What is the difference between process.nextTick and setImmediate?

> `process.nextTick`: runs callback at end of current operation, before event loop continues — highest priority microtask. `setImmediate`: runs in the check phase of the next event loop iteration. `nextTick` always fires before `setImmediate`.

Q13. What are Node.js streams?

> Streams are abstract interfaces for handling data chunk by chunk. Types: Readable (read data), Writable (write data), Duplex (both), Transform (modify data). Used for: file I/O, HTTP requests, compression. More memory efficient than loading entire files.

Q14. What is the cluster module in Node.js?

> The `cluster` module allows creating child processes (workers) that share the same server port. Each worker runs on a separate CPU core, bypassing the GIL limitation. The master process manages workers and distributes incoming connections.

Q15. What is the difference between EventEmitter and Streams?

> EventEmitter is the base class for event-driven programming — emit/listen to named events. Streams extend EventEmitter and add structured data flow semantics (readable/writable interfaces, backpressure). All streams are EventEmitters.

Q16. How does Node.js handle CPU-intensive tasks?

> Options: Worker Threads (`worker_threads` module — share memory, good for CPU work), Child Processes (`child_process.fork()`), Cluster module, offloading to external services, or using native C++ addons.

Q17. What is middleware pattern in Node.js?

> Middleware pattern chains functions where each can process a request and pass to the next. Popularized by Express/Koa. Custom implementation: array of functions, each called with `(req, res, next)` — enables composable, reusable request processing.

Q18. What is the purpose of the buffer module in Node.js?

> Buffer handles raw binary data (TCP streams, file operations). `Buffer.from()`, `Buffer.alloc()`. Buffers are fixed-size allocations outside V8 heap. Used before Streams API was available; now often used together with streams.

Q19. How do you handle uncaught exceptions in Node.js?

> `process.on('uncaughtException', handler)` catches sync errors. `process.on('unhandledRejection', handler)` catches unhandled Promise rejections. Best practice: log the error and exit gracefully (restart with PM2). Never silently ignore.

Q20. What is package-lock.json?

> `package-lock.json` locks the exact version of every installed package (including transitive dependencies). Ensures consistent installs across environments. Committed to version control. Never manually edit.

HARD (10 QUESTIONS)

Q21. Explain Node.js's libuv and its thread pool.

> libuv is a C library providing the event loop, async I/O, timers, and a thread pool. The thread pool (default 4 threads, max 1024) handles blocking operations: file system, DNS lookups, crypto. Network I/O is handled by the OS directly (epoll/kqueue).

Q22. What is backpressure in Node.js streams?

> Backpressure occurs when a writable stream's buffer is full and the readable stream produces data faster than it can be consumed. Signals via `write()` returning false and `drain` event. `pipe()` handles backpressure automatically.

Q23. How does garbage collection work in V8 (Node.js)?

> V8 uses generational GC: new space (Scavenger — young gen, minor GC, frequent), old space (Mark-Sweep/Mark-Compact — major GC, less frequent). New objects start in new space; surviving multiple GC cycles move to old space.

Q24. What is N-API (now Node-API)?

> Node-API is a stable C API for building native addons for Node.js. It abstracts V8/libuv, so addons work across Node.js versions without recompilation. Used for performance-critical C/C++ code called from JavaScript.

Q25. How do you implement caching in Node.js?

> Options: in-memory (plain object, Map, node-cache), Redis (`ioredis` or `redis` client), CDN cache, HTTP caching headers (Cache-Control, ETag). Module-level caching: `require()` caches modules after first load.

Q26. What is the difference between fork, spawn, and exec in child_process?

> `exec`: runs shell command, buffers output (small data). `spawn`: streams output, no shell, better for large data/long-running. `fork`: special spawn for Node.js modules — creates IPC channel for message passing between parent and child.

Q27. How does module caching work in Node.js?

> After first `require()`, the module is cached in `require.cache` (keyed by resolved filename). Subsequent `require()` calls return the cached exports object without re-executing. To force reload: delete from `require.cache` (use carefully).

Q28. What is the Async Hooks API in Node.js?

> `async_hooks` tracks async operations through their lifecycle (init, before, after, destroy, promiseResolve). Used to propagate context (request IDs, traces) through async call chains — basis for distributed tracing and APM in Node.js.

Q29. How would you diagnose memory leaks in a Node.js application?

> Tools: `node --inspect` + Chrome DevTools heap snapshots, `clinic.js heap`, `heapdump` package. Patterns to look for: global variables growing, event listeners not removed, closures retaining large objects, circular references in old-space.

Q30. What is the Worker Threads module and when to use it over the Cluster module?

> Worker Threads: share memory (SharedArrayBuffer, Atomics), lighter weight, good for parallel CPU computation within a single server. Cluster: separate processes, separate memory, full isolation, fault tolerant, good for handling more HTTP connections. Use Workers for math/crypto; Cluster for scaling server throughput.

---

8. MySQL & PostgreSQL

Subtopics: SQL Queries & Joins | Indexing & Optimization | Transactions & Advanced

---

EASY (10 QUESTIONS)

Q1. What is the difference between MySQL and PostgreSQL?

> MySQL: faster for read-heavy workloads, widely used, simpler. PostgreSQL: ACID-compliant, supports advanced types (JSON, arrays, custom types), better for complex queries, supports full-text search, more standards-compliant.

Q2. What are the types of SQL commands?

> DDL (Data Definition): CREATE, ALTER, DROP, TRUNCATE. DML (Data Manipulation): SELECT, INSERT, UPDATE, DELETE. DCL (Data Control): GRANT, REVOKE. TCL (Transaction Control): COMMIT, ROLLBACK, SAVEPOINT.

Q3. What is a PRIMARY KEY?

> A primary key uniquely identifies each row in a table. It cannot be NULL and must be unique. A table can have only one primary key (can be composite).

Q4. What is the difference between WHERE and HAVING?

> WHERE filters rows before grouping (used without GROUP BY). HAVING filters groups after GROUP BY aggregation. WHERE cannot use aggregate functions; HAVING can.

Q5. What are the types of SQL JOINs?

> INNER JOIN: rows where condition matches in both tables. LEFT JOIN: all rows from left + matching from right (NULL if no match). RIGHT JOIN: opposite of LEFT. FULL OUTER JOIN: all rows from both (NULLs for non-matches). CROSS JOIN: cartesian product.

Q6. What is a FOREIGN KEY?

> A foreign key is a column that references the primary key of another table. It enforces referential integrity — you cannot insert a value that doesn't exist in the referenced table.

Q7. What is the difference between CHAR and VARCHAR?

> CHAR(n): fixed-length, always uses n bytes (padded with spaces). VARCHAR(n): variable-length, uses only as much space as needed (+1-2 bytes overhead). Use CHAR for fixed-size data (country codes); VARCHAR for variable-size strings.

Q8. What is a NULL value in SQL?

> NULL represents the absence of a value (unknown). NULL != NULL — use `IS NULL` or `IS NOT NULL`. NULL propagates in arithmetic (5 + NULL = NULL). Use `COALESCE(col, default)` to substitute a default.

Q9. What is the difference between DELETE, TRUNCATE, and DROP?

> DELETE: removes specific rows (with WHERE), logged, can rollback. TRUNCATE: removes all rows, faster, minimally logged, resets auto-increment, cannot rollback in MySQL. DROP: removes the entire table structure and data.

Q10. What is an INDEX in SQL?

> An index is a data structure that speeds up data retrieval. It works like a book index — stores pointer to row based on column value. Speeds up SELECT but slows INSERT/UPDATE/DELETE. Default is B-tree index.

MEDIUM (10 QUESTIONS)

Q11. What is normalization? Explain 1NF, 2NF, 3NF.

> Normalization organizes data to reduce redundancy. 1NF: atomic values, no repeating groups. 2NF: 1NF + no partial dependency (non-key columns depend on full primary key). 3NF: 2NF + no transitive dependency (non-key columns depend only on primary key).

Q12. What is the difference between UNION and UNION ALL?

> UNION removes duplicate rows from the combined result. UNION ALL includes all rows including duplicates. UNION ALL is faster as it skips deduplication. Both require same number of columns with compatible types.

Q13. What are window functions in SQL?

> Window functions perform calculations across a set of rows related to the current row without collapsing them. Functions: ROW_NUMBER(), RANK(), DENSE_RANK(), LAG(), LEAD(), SUM() OVER(), AVG() OVER(). Use OVER(PARTITION BY ... ORDER BY ...).

Q14. What is a subquery vs a JOIN — when to use each?

> Subquery: nested query in WHERE/FROM/SELECT. Good for single-value lookups or existence checks. JOIN: combines rows from multiple tables. JOINs are generally faster for large datasets as optimizer can use indexes. Subqueries are more readable for complex logic.

Q15. What is an EXPLAIN/EXPLAIN ANALYZE in SQL?

> EXPLAIN shows the query execution plan without running the query. EXPLAIN ANALYZE runs it and shows actual vs estimated stats. Use to identify: full table scans, missing indexes, expensive sorts, nested loop vs hash joins.

Q16. What are ACID properties in databases?

> Atomicity: all-or-nothing (transaction fully completes or fully fails). Consistency: database always moves from one valid state to another. Isolation: concurrent transactions don't interfere. Durability: committed data survives system failures.

Q17. What is the difference between clustered and non-clustered indexes?

> Clustered index: determines physical row storage order — there can be only one per table (InnoDB primary key is clustered). Non-clustered index: separate structure pointing to rows — multiple allowed, faster lookups but extra storage/indirection.

Q18. What are stored procedures and when to use them?

> Stored procedures are precompiled SQL code stored in the database. Benefits: reduced network traffic, reusable logic, better performance (pre-parsed). Downsides: harder to version control, business logic in DB couples DB to app. Use for batch operations or when DB logic is critical.

Q19. What is a transaction isolation level?

> Isolation levels (weakest to strongest): READ UNCOMMITTED (dirty reads), READ COMMITTED (default in PG — no dirty reads), REPEATABLE READ (default in MySQL InnoDB — no phantom reads for same query), SERIALIZABLE (full isolation, slowest).

Q20. What is the N+1 query problem?

> N+1 occurs when fetching N records and then making 1 additional query per record (e.g., fetch all users, then query each user's orders). Fix with JOIN, eager loading, or batch queries. Common in ORMs — use `include` / `eager_load`.

HARD (10 QUESTIONS)

Q21. How does InnoDB implement MVCC (Multi-Version Concurrency Control)?

> InnoDB stores multiple versions of each row using undo logs. Each row has hidden columns: DB_TRX_ID (transaction that last modified), DB_ROLL_PTR (pointer to undo log). Reads see a consistent snapshot based on transaction start time — no read locks needed.

Q22. What is a covering index?

> A covering index contains all columns needed by a query, so the DB can answer the query using only the index without accessing the actual table rows (avoids table heap/clustered index lookup). Results in "Index Only Scan" in EXPLAIN.

Q23. Explain the difference between PostgreSQL's JSONB and JSON types.

> JSON: stores text as-is (preserves whitespace, key order, duplicates). JSONB: stores parsed binary format (no whitespace, deduplicates keys, reorders keys). JSONB supports GIN indexing for fast queries; JSON does not. Use JSONB for querying, JSON for logging exact input.

Q24. What is connection pooling and why is it needed?

> Creating DB connections is expensive (TCP handshake, auth, memory). Connection pooling maintains a pool of reusable connections. Libraries: PgBouncer (PostgreSQL), `mysql2` pool. Configured with min/max connections. Prevents connection exhaustion under high concurrency.

Q25. How do you handle database migrations in production?

> Use migration tools (Flyway, Liquibase, Alembic, Knex migrations). Principles: backward-compatible changes (add columns, not remove), blue-green deployment compatibility, zero-downtime: add nullable column → backfill → add constraint → remove old column. Never edit applied migrations.

Q26. What is table partitioning in PostgreSQL?

> Partitioning divides a large table into smaller physical pieces (partitions) while appearing as one table. Types: Range (date ranges), List (categorical values), Hash (distributed by hash). Improves query performance (partition pruning) and maintenance (archive old partitions).

Q27. What is the difference between OLTP and OLAP databases?

> OLTP (Online Transaction Processing): many small, fast read/write transactions, normalized schema, row-oriented storage (MySQL, PostgreSQL). OLAP (Online Analytical Processing): few large analytical queries, denormalized/star schema, column-oriented storage (Redshift, BigQuery, ClickHouse).

Q28. How does PostgreSQL's WAL (Write-Ahead Log) work?

> Before any data file is modified, PostgreSQL writes the change to WAL (sequential, append-only). On crash, WAL is replayed to recover. WAL enables: crash recovery, streaming replication (standby reads WAL), point-in-time recovery (PITR), and logical replication.

Q29. What are CTE (Common Table Expressions) and recursive CTEs?

> CTE (`WITH query AS (SELECT ...)`) creates a named temporary result reusable in the query. Recursive CTE uses `WITH RECURSIVE` — base case + recursive part that references itself. Used for tree/hierarchy traversal (org charts, file systems, BOM).

Q30. How do you optimize a slow query in PostgreSQL?

> Steps: EXPLAIN ANALYZE to find bottleneck, check for sequential scans (add index), check join strategy (enable/disable hashjoin/nestloop), update table statistics (ANALYZE), add covering indexes, rewrite subqueries as JOINs, use materialized views for complex aggregations, partition large tables.

---

9. Postman

Subtopics: API Testing & Collections | Variables & Environments | Automation & Scripts

---

EASY (10 QUESTIONS)

Q1. What is Postman?

> Postman is an API platform for building, testing, and documenting APIs. It provides a GUI to send HTTP requests, inspect responses, create collections, run automated tests, and manage API environments.

Q2. What is a Postman Collection?

> A collection is a group of related API requests organized in folders. It can be shared, run via Collection Runner, or automated with Newman (CLI runner).

Q3. What are Postman Environments?

> Environments store sets of variables (key-value pairs) that can be switched. Example: Dev environment has `baseUrl = http://localhost:3000`, Prod has `baseUrl = https://api.example.com`. Variables referenced as `{{baseUrl}}`.

Q4. What are the types of variables in Postman?

> Scope (narrowest to widest): Data → Local → Environment → Collection → Global. Narrower scope overrides wider. Use environment for URL/tokens; global for truly shared values.

Q5. How do you send a POST request with a JSON body in Postman?

> Select POST method → Body tab → raw → select JSON from dropdown → enter JSON. Set Content-Type header to `application/json` (auto-set when selecting JSON).

Q6. What is Pre-request Script in Postman?

> JavaScript code that runs before a request is sent. Used to: set variables dynamically (timestamps, tokens), generate signatures, modify headers/body before sending.

Q7. What are Tests in Postman?

> Tests are JavaScript code in the "Tests" tab that run after a response is received. Use `pm.test()` and `pm.expect()` assertions to validate: status code, response body, headers, response time.

Q8. How do you authenticate requests in Postman?

> Use the Authorization tab: select type (Bearer Token, Basic Auth, API Key, OAuth 2.0). For Bearer: enter JWT token. Postman adds the appropriate headers automatically.

Q9. What is the Collection Runner in Postman?

> Collection Runner executes all requests in a collection sequentially. Supports data files (CSV/JSON) for parameterized testing, setting iterations, delays, and viewing results with pass/fail status.

Q10. What is Newman?

> Newman is Postman's command-line collection runner. Allows running collections in CI/CD pipelines: `newman run collection.json -e environment.json`. Supports multiple reporters (CLI, JSON, HTML, JUnit).

MEDIUM (10 QUESTIONS)

Q11. How do you chain requests in Postman (use response from one request in another)?

> In the Tests script of the first request: `pm.environment.set("userId", pm.response.json().id)`. In the second request URL/body: use `{{userId}}`. Collection Runner/Newman executes them in order.

Q12. What is the difference between pm.environment and pm.globals in Postman?

> `pm.environment.set/get`: scoped to current environment (switch environments = different values). `pm.globals.set/get`: application-wide, persist across environments. Environment variables override globals.

Q13. How do you write a test to validate a JSON response schema in Postman?

> Use `pm.test` with `pm.response.json()`: check required fields, types, values. For schema validation: use `ajv` library or Postman's built-in `tv4`. Example: `pm.expect(jsonData).to.have.property('id').that.is.a('number')`.

Q14. How do you handle dynamic tokens (auto-refresh) in Postman?

> In the Collection's Pre-request Script: check if token exists and is valid. If expired/missing, send login request using `pm.sendRequest()`, extract token from response, store in environment variable. All collection requests share this token.

Q15. What is Postman Mock Server?

> Postman Mock Server creates a simulated API from a collection's saved examples. Returns example responses based on request matching. Useful for frontend development before backend is ready or for testing without live APIs.

Q16. How do you use data files in Collection Runner?

> Prepare CSV or JSON file with test data. In Collection Runner, select the data file. Variables from the file are available as `{{variableName}}` in requests. Each row/object runs as a separate iteration.

Q17. What is `pm.sendRequest()` and when to use it?

> `pm.sendRequest(options, callback)` sends an HTTP request programmatically from Pre-request Script or Tests. Used for: getting auth tokens, seeding test data before a request, chaining-related API calls within a single request's lifecycle.

Q18. How do you test for response time in Postman?

> `pm.test("Response time < 200ms", () => { pm.expect(pm.response.responseTime).to.be.below(200) })`. Use in Tests tab. Helps enforce performance SLAs.

Q19. What is the difference between form-data and x-www-form-urlencoded in Postman?

> `form-data`: supports file uploads, multipart encoding — each field/file is a separate part. `x-www-form-urlencoded`: URL-encoded key-value pairs in body (like URL query string) — no file upload support. Choose form-data for file uploads.

Q20. How do you import/export collections in Postman?

> Export: Collections tab → right-click → Export → choose format (Collection v2.1 JSON). Import: Import button → select JSON file/URL/Postman link. Collections can also be shared via Postman workspace.

HARD (10 QUESTIONS)

Q21. How do you integrate Postman/Newman into a CI/CD pipeline?

> Export collection + environment JSON to repo. In CI (GitHub Actions/Jenkins): `npm install -g newman`, run `newman run collection.json -e env.json --reporters cli,junit --reporter-junit-export results.xml`. Parse JUnit XML for test results.

Q22. What is Postman Flows?

> Postman Flows is a visual, no-code tool for building API workflows. Drag-and-drop blocks for requests, conditions, loops, and data transformation. Useful for prototyping complex API orchestration without writing code.

Q23. How do you implement OAuth 2.0 flow in Postman?

> Authorization tab → OAuth 2.0 → configure: Auth URL, Token URL, Client ID/Secret, Scope. Click "Get New Access Token" — Postman opens browser for authorization, exchanges code for token, stores it. Token can be set at collection level for all requests.

Q24. What are Postman Monitors?

> Monitors run collections on a schedule (cron-like) to check API uptime and performance. They run in Postman's cloud infrastructure. Reports pass/fail status, response times. Useful for production API health monitoring.

Q25. How do you manage secrets (API keys, passwords) in Postman without exposing them?

> Use Postman Vault (encrypted, not synced to cloud) for sensitive values. Set as environment variables — never hardcode in requests or collection. Use `.postman_environment.json.example` files with placeholders in version control, not actual secrets.

Q26. How does Postman handle SSL certificate issues for development?

> Settings → SSL certificate verification → OFF for testing self-signed certs. For production-like testing: install custom CA certificate in Postman Settings → Certificates → add CA certificate file. Never disable SSL verification in production testing.

Q27. What is the difference between Collection v2 and v2.1 format?

> v2.1 is the current standard. Differences: v2.1 uses UUID-based IDs, better handling of variables, scripts, and auth. v2.1 is required for most advanced Postman features. Always export as v2.1 for compatibility with Newman and latest Postman.

Q28. How do you generate API documentation from Postman?

> Add descriptions to requests, examples, and parameters in Postman. Publish collection as documentation (Collections → View Documentation → Publish). Generates interactive web docs. Can also export to OpenAPI/Swagger format using Postman's API schema feature.

Q29. How do you handle pagination testing in Postman?

> Use environment variables to track page number (`{{page}}`). In Tests: extract next page cursor/number from response, set environment variable. Use Collection Runner with a loop or Postman Flows for automated pagination traversal.

Q30. What is Postman API Builder and how does it relate to OpenAPI?

> Postman API Builder allows defining API schema in OpenAPI 3.0/Swagger format directly in Postman. From the schema, you can generate collections, mock servers, and documentation. It enables design-first API development with automatic test generation.

---

10. RAG (Retrieval Augmented Generation)

Subtopics: Architecture & Embeddings | Vector Databases | Chunking & Retrieval

---

EASY (10 QUESTIONS)

Q1. What is RAG?

RAG (Retrieval Augmented Generation) is a technique that enhances LLM responses by retrieving relevant documents from an external knowledge base and including them in the prompt context. Reduces hallucinations and enables up-to-date answers.

Q2. What are embeddings?

Embeddings are dense numerical vector representations of text (or other data). Semantically similar text produces similar vectors. Generated by models like OpenAI `text-embedding-ada-002`, Sentence-BERT. Used to measure text similarity via cosine distance.

Q3. What is a vector database?

A vector database stores and indexes high-dimensional vectors (embeddings) for fast similarity search. Examples: Pinecone, Weaviate, Chroma, FAISS, Milvus, pgvector. Supports ANN (Approximate Nearest Neighbor) queries.

Q4. What are the two main phases of RAG?

1. Indexing (offline): document loading → chunking → embedding → storing in vector DB. 2. Retrieval (online): embed query → similarity search in vector DB → retrieve top-k chunks → inject into LLM prompt.

Q5. What is chunking in RAG?

Chunking splits large documents into smaller pieces before embedding. Necessary because: embedding models have token limits, and smaller focused chunks improve retrieval precision. Common strategies: fixed-size, sentence, paragraph, semantic.

Q6. What is cosine similarity?

> Cosine similarity measures the cosine of the angle between two vectors, ranging from -1 to 1. Value of 1 means identical direction (very similar), 0 means orthogonal (unrelated). Used to find most similar document chunks to a query.

Q7. Why does RAG reduce hallucinations?

> By grounding the LLM's response in retrieved factual documents, RAG provides specific context the model should use. The model is instructed to answer based on the provided documents rather than relying solely on its training data.

Q8. What is the difference between RAG and fine-tuning?

> Fine-tuning: updates model weights on domain-specific data — knowledge is baked in. RAG: retrieves external knowledge at inference time — no weight updates needed, easily updated. RAG is better for dynamic/changing knowledge; fine-tuning for style/format.

Q9. What is a retriever in RAG?

> A retriever is the component that finds and returns the most relevant documents/chunks from the knowledge base given a query. Types: vector similarity retriever, BM25 (keyword-based), hybrid (combining both).

Q10. What is top-k in RAG retrieval?

> Top-k is the number of most similar chunks to retrieve and pass to the LLM. Typically k=3-10. Higher k gives more context but increases prompt length and cost. The optimal k depends on chunk size and task complexity.

MEDIUM (10 QUESTIONS)

Q11. What is the difference between sparse and dense retrieval?

> Sparse retrieval (BM25, TF-IDF): keyword-based, exact term matching, interpretable, no semantic understanding. Dense retrieval: embedding-based, semantic similarity, handles paraphrasing. Hybrid combines both for better results.

Q12. What is Hybrid Search in RAG?

> Hybrid search combines sparse (BM25) and dense (vector) retrieval using Reciprocal Rank Fusion (RRF) or weighted scoring. Gets the best of both: keyword precision (sparse) + semantic understanding (dense). Often outperforms either alone.

Q13. What is a reranker in RAG?

> A reranker is a cross-encoder model that takes (query, chunk) pairs and scores their relevance more accurately than embedding similarity alone. Applied after initial retrieval to reorder the top-k results. Examples: Cohere Rerank, BGE reranker. Slower but more accurate.

Q14. What is the context window problem in RAG?

> LLMs have limited context windows. If retrieved chunks + query exceeds the window, some content is truncated. Solutions: smaller chunks, selecting fewer chunks, summarizing retrieved content, using LLMs with larger context windows (GPT-4-turbo 128k, Claude 200k).

Q15. What is Multi-Query RAG?

> Multi-Query RAG generates multiple rephrasings of the original query using the LLM, retrieves chunks for each variant, and deduplicates results. Increases recall by capturing chunks that match any of the query variations.

Q16. What are the challenges of chunking strategies?

> Fixed-size: may split sentences/paragraphs mid-thought. Sentence: variable chunk sizes. Semantic: computationally expensive. Recursive character splitting: handles multiple separators. The right strategy depends on document structure (code, text, tables differ).

Q17. What is a document store vs vector store in RAG?

> Vector store: stores embeddings and original text for similarity search. Document store: stores the full original documents for retrieval/reading. Often combined — vector DB returns IDs, fetch full text from document store. Separation improves scalability.

Q18. What is the difference between RAG and long-context LLMs?

> Long-context LLMs (GPT-4-turbo, Gemini 1.5 Pro, Claude) can process entire documents in context. RAG retrieves only relevant chunks. RAG is better for large corpora (millions of docs), cost efficiency. Long-context is better for small corpora where full context matters.

Q19. What is knowledge graph RAG?

> Knowledge Graph RAG stores entities and relationships in a graph (Neo4j) in addition to text chunks. Enables structured reasoning over relationships. GraphRAG (Microsoft) uses community detection and graph traversal for better global understanding.

Q20. What is contextual compression in RAG?

> Contextual compression further processes retrieved chunks: extract only the relevant sentences (LLMChainExtractor), or filter out irrelevant chunks entirely (LLMChainFilter). Reduces noise in the context and improves answer quality.

HARD (10 QUESTIONS)

Q21. How do you evaluate a RAG system?

> Metrics: Faithfulness (is the answer grounded in retrieved docs?), Answer Relevance (does it answer the question?), Context Precision (are retrieved chunks relevant?), Context Recall (are all relevant chunks retrieved?). Frameworks: RAGAS, TruLens, DeepEval.

Q22. What is Self-RAG?

> Self-RAG is a framework where the LLM decides when to retrieve (retrieval token), critiques retrieved passages (relevance/support tokens), and evaluates its own generation (utility token). Enables adaptive retrieval — only retrieving when necessary.

Q23. What is the problem of lost in the middle in RAG?

> LLMs tend to give less attention to information in the middle of long contexts, performing better with information at the beginning/end. Solution: sort retrieved chunks by relevance (most relevant first/last), use rerankers, or use models explicitly trained to handle long contexts.

Q24. What is Agentic RAG?

> Agentic RAG uses an AI agent to orchestrate multi-step retrieval: the agent can decide to search multiple times, refine queries, use different tools (web search + vector DB), verify retrieved information, and synthesize across multiple retrievals before answering.

Q25. How do you handle tables and structured data in RAG?

> Challenges: tables lose structure when converted to text. Strategies: convert to natural language summaries, use specialized table embeddings, store in structured DB and use Text-to-SQL, extract to JSON/CSV with schema description, use multimodal models for visual table parsing.

Q26. What is parent-child chunking in RAG?

> Parent-child chunking stores small child chunks for precise retrieval and larger parent chunks for full context. At retrieval: find relevant child chunks → fetch the corresponding parent chunk for richer context. Balances retrieval precision with context completeness.

Q27. How do you handle multimodal RAG (images, PDFs with figures)?

> Multimodal embedding models (CLIP, OpenAI multimodal) embed images and text in shared space. PDFs: extract text + images separately, embed both. At retrieval: combine results from text and image retrievers. LLM must be multimodal to use image context.

Q28. What is RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval)?

> RAPTOR recursively clusters and summarizes documents at multiple levels of abstraction, building a tree. Retrieval at multiple levels: leaf nodes (detailed) and higher nodes (summaries). Better for questions requiring global understanding across many documents.

Q29. How do you prevent prompt injection attacks in RAG systems?

> Techniques: separate retrieved content from instructions (clear tags), instruct model to only use retrieved context, sanitize retrieved content, use constitutional filters, apply output validation, maintain trust boundaries between document content and system instructions.

Q30. What is the difference between online and offline RAG pipelines?

> Offline (indexing): periodic batch jobs that crawl, chunk, embed, and upsert into vector DB. Online (retrieval): real-time query embedding, search, and generation. Hybrid: offline for static docs, real-time ingestion for live data. Incremental updates needed when documents change.

---

11. Git & GitHub

Subtopics: Git Core Commands | Branching & Merging | GitHub Collaboration

---

EASY (10 QUESTIONS)

Q1. What is Git?

> Git is a distributed version control system that tracks changes to files over time. Every developer has a full copy of the repository including history. Allows collaboration, branching, merging, and reverting changes.

Q2. What is the difference between git fetch and git pull?

> `git fetch`: downloads changes from remote but does NOT merge into local branch. `git pull`: fetch + merge (or rebase) in one step. Use fetch to review changes before integrating.

Q3. What is a branch in Git?

> A branch is a lightweight pointer to a specific commit. It allows working on different features/fixes in isolation. Default branch is `main` (or `master`). Create: `git branch feature-x`. Switch: `git checkout feature-x` or `git switch feature-x`.

Q4. What is the difference between git merge and git rebase?

> `git merge`: creates a merge commit combining two branches, preserves full history. `git rebase`: moves/replays commits on top of another branch, creating a linear history. Rebase is cleaner but rewrites history — don't rebase public/shared branches.

Q5. What is `git stash`?

> `git stash` temporarily saves uncommitted changes (staged and unstaged) so you can switch branches or pull updates without committing. Restore with `git stash pop`. List with `git stash list`.

Q6. What is a commit in Git?

> A commit is a snapshot of the project at a point in time. Each commit has a SHA hash, author, timestamp, message, and pointer to parent commit(s). Created with `git commit -m "message"`.

Q7. What is the staging area (index) in Git?

> The staging area is a preparation zone between the working directory and the repository. `git add` moves changes to the staging area. `git commit` saves staged changes as a commit. Allows choosing which changes to commit.

Q8. How do you undo the last commit in Git?

> `git reset --soft HEAD~1`: undo commit, keep changes staged. `git reset --mixed HEAD~1`: undo commit, keep changes unstaged. `git reset --hard HEAD~1`: undo commit and discard all changes. For pushed commits, prefer `git revert HEAD`.

Q9. What is `.gitignore`?

> `.gitignore` lists files and patterns that Git should not track (e.g., `node_modules/`, `.env`, `*.log`). Patterns support wildcards, negation (`!`), and directories. Existing tracked files are not affected — use `git rm --cached` to untrack.

Q10. What is a pull request (PR)?

> A pull request is a GitHub (or GitLab/Bitbucket) feature to propose merging changes from one branch to another. It enables code review, discussion, automated CI checks, and approval workflow before merging.

MEDIUM (10 QUESTIONS)

Q11. What is the difference between git revert and git reset?

> `git reset`: moves HEAD backward, removes commits from history (dangerous for shared branches). `git revert`: creates a NEW commit that undoes a previous commit's changes — safe for shared branches as it preserves history.

Q12. What is git cherry-pick?

> `git cherry-pick <commit-hash>` applies a specific commit from one branch onto the current branch without merging the entire branch. Useful for applying bug fixes from one branch to another.

Q13. What is a merge conflict and how do you resolve it?

> A merge conflict occurs when two branches modify the same lines of a file differently. Git marks conflicts with `<<<<<<<`, `=======`, `>>>>>>>`. Resolve by editing the file to keep desired changes, then `git add` the file and complete the merge with `git commit`.

Q14. What is git rebase -i (interactive rebase)?

> Interactive rebase lets you modify commit history: squash multiple commits into one, reword messages, reorder, edit, or drop commits. Used before pushing to clean up work-in-progress commits: `git rebase -i HEAD~3`.

Q15. What is a git tag and when to use it?

> Tags mark specific commits as important (e.g., release versions). Lightweight tag: just a pointer. Annotated tag: stores metadata (tagger, date, message). Create: `git tag v1.0.0`. Push: `git push origin v1.0.0`. Used for versioned releases.

Q16. What is the difference between git clone and git fork?

> Clone: creates a local copy of any repository. Fork (GitHub concept): creates your own copy of someone else's repository on GitHub, enabling you to make changes and submit PRs. Clone is Git; fork is a GitHub feature.

Q17. What are GitHub Actions?

> GitHub Actions is a CI/CD platform integrated with GitHub. Workflows defined in `.github/workflows/*.yml` files. Triggered by events (push, PR, schedule). Runs jobs in containers — build, test, deploy, lint. Free for public repos.

Q18. What is the Git flow branching strategy?

> Git flow: `main` (production), `develop` (integration), `feature/*` branches from develop, `release/*` for release prep, `hotfix/*` from main for urgent fixes. More structured but complex. Alternatives: GitHub flow (simpler), trunk-based development.

Q19. What is `git bisect`?

> `git bisect` uses binary search to find the commit that introduced a bug. `git bisect start` → `git bisect bad` (current broken) → `git bisect good <hash>` (last known good) → Git checks out midpoint → mark good/bad → repeats until culprit is found.

Q20. What is a GitHub webhook?

> A webhook is an HTTP POST sent by GitHub to a URL you specify when certain events occur (push, PR, issue). Used to trigger CI/CD pipelines, notify Slack, update deployment systems, or trigger any external automation.

HARD (10 QUESTIONS)

Q21. How does Git store data internally?

> Git stores data as objects in `.git/objects`: Blob (file content), Tree (directory structure → blobs/trees), Commit (tree + metadata + parent hash), Tag. Objects are SHA-1 hashed. Branch files (`.git/refs`) contain just a commit hash. Everything is content-addressed.

Q22. What is the difference between git merge --ff, --no-ff, and --squash?

> `--ff` (fast-forward): moves branch pointer forward if no divergence (no merge commit). `--no-ff`: always creates merge commit preserving branch topology. `--squash`: combines all branch commits into one staged change (no merge commit, not linked to source branch).

Q23. How do you protect the main branch in GitHub?

> Branch protection rules: require PR reviews, require status checks (CI) to pass, disable force push, require signed commits, restrict who can push, require linear history. Set in repository Settings → Branches.

Q24. How does Git handle binary files vs text files in merging?

> Git merges text files line-by-line and can auto-resolve non-conflicting changes. Binary files cannot be merged — last write wins or conflict occurs. Use `.gitattributes` to specify binary files (`*.png binary`), define custom merge drivers, or use Git LFS for large binaries.

Q25. What is Git LFS (Large File Storage)?

> Git LFS replaces large files (images, videos, models) in the repo with small text pointers. Actual files are stored on a separate LFS server. Requires `git lfs install` and `git lfs track "*.psd"`. Prevents repo bloat.

Q26. How do you resolve a detached HEAD state in Git?

> Detached HEAD means HEAD points directly to a commit, not a branch. To keep changes: `git checkout -b new-branch` to create a branch at current commit. To discard: `git checkout main`. Occurs after `git checkout <commit-hash>` or `git bisect`.

Q27. How do GitHub Actions secrets work?

> Secrets are encrypted environment variables stored at repository/org level. Set in Settings → Secrets → Actions. Accessed in workflows as `${{ secrets.SECRET_NAME }}`. Never logged in output. Scoped to environments for production protection.

Q28. What is the difference between a shallow clone and a full clone?

> Shallow clone (`git clone --depth 1`): downloads only the latest commit and its tree, not full history. Faster and smaller. Full clone: downloads complete history. Use shallow for CI/CD builds; full for development where you need history, blame, bisect.

Q29. How do you sign commits in Git with GPG?

> Generate GPG key, add public key to GitHub account. Configure Git: `git config --global user.signingkey KEY_ID`, `git config --global commit.gpgsign true`. Commits signed with private key; GitHub shows "Verified" badge. Ensures commit authenticity.

Q30. What is the CODEOWNERS file in GitHub?

> `.github/CODEOWNERS` maps file patterns to GitHub users/teams who are automatically requested for review when those files change in a PR. Ensures relevant team members review changes to critical files. Works with branch protection required reviews.

---

12. AES & Computer Networks

Subtopics: AES Encryption | Network Protocols (OSI/TCP) | Security & Application Layer

---

EASY (10 QUESTIONS)

Q1. What is AES?

> AES (Advanced Encryption Standard) is a symmetric block cipher adopted by NIST in 2001. It replaced DES. Uses the same key for encryption and decryption. Key sizes: 128, 192, or 256 bits. Block size: always 128 bits.

Q2. What is the OSI model?

> OSI (Open Systems Interconnection) has 7 layers: 1-Physical (bits, cables), 2-Data Link (MAC, frames), 3-Network (IP, routing), 4-Transport (TCP/UDP, ports), 5-Session (sessions), 6-Presentation (encoding, SSL/TLS), 7-Application (HTTP, DNS, FTP).

Q3. What is the difference between TCP and UDP?

> TCP: connection-oriented, reliable (guaranteed delivery, ordering), flow control, slower. UDP: connectionless, unreliable (no delivery guarantee), faster, lower overhead. TCP for web/files; UDP for streaming, gaming, DNS.

Q4. What is an IP address?

> An IP address is a numerical label assigned to each device on a network. IPv4: 32-bit (e.g., 192.168.1.1). IPv6: 128-bit (e.g., 2001:db8::1). Used for network-layer routing.

Q5. What is DNS?

> DNS (Domain Name System) translates human-readable domain names (google.com) to IP addresses. Hierarchy: Root servers → TLD (.com, .org) → Authoritative servers. Resolvers cache results for TTL duration.

Q6. What is the difference between HTTP and HTTPS?

> HTTP: plaintext, port 80, no encryption. HTTPS: HTTP over TLS (Transport Layer Security), port 443, encrypted, authenticates server via certificate. All data is encrypted, preventing eavesdropping and MITM attacks.

Q7. What is a MAC address?

> MAC (Media Access Control) address is a 48-bit hardware identifier assigned to a network interface. Used for communication within a local network (Layer 2). Format: 6 pairs of hex (e.g., AA:BB:CC:DD:EE:FF). Burned into NIC but can be spoofed.

Q8. What is a subnet mask?

> A subnet mask divides an IP address into network and host portions. Example: 255.255.255.0 (/24) means first 24 bits are network. Used in subnetting to divide networks into smaller segments.

Q9. What is symmetric vs asymmetric encryption?

> Symmetric: same key for encryption and decryption (AES, DES) — fast, used for bulk data. Asymmetric: public key encrypts, private key decrypts (RSA, ECC) — slower, used for key exchange and digital signatures.

Q10. What is a firewall?

> A firewall monitors and controls incoming/outgoing network traffic based on security rules. Types: Packet filtering (Layer 3/4), Stateful inspection, Application firewall (WAF — Layer 7). Can be hardware or software.

MEDIUM (10 QUESTIONS)

Q11. Explain the AES encryption process.

> AES processes 128-bit blocks through rounds: 128-bit key = 10 rounds, 192-bit = 12 rounds, 256-bit = 14 rounds. Each round: SubBytes (S-box substitution), ShiftRows (row shift), MixColumns (column mixing), AddRoundKey (XOR with round key). Last round omits MixColumns.

Q12. What are AES modes of operation?

> ECB (Electronic Codebook): same plaintext → same ciphertext (insecure, reveals patterns). CBC (Cipher Block Chaining): XOR with previous ciphertext block (needs IV). CTR (Counter): stream cipher mode, parallelizable. GCM (Galois/Counter Mode): CTR + authentication tag (AEAD — authenticated encryption). Use GCM for modern applications.

Q13. What is the three-way handshake in TCP?

> SYN: client sends SYN to server. SYN-ACK: server responds with SYN-ACK. ACK: client sends ACK. Connection is established. Ensures both parties can send/receive before data transfer. Termination uses four-way FIN-ACK sequence.

Q14. What is TLS and how does the TLS handshake work?

> TLS (Transport Layer Security) provides encryption and authentication over TCP. Handshake: Client Hello (cipher suites, TLS version) → Server Hello + Certificate → Client verifies cert → Key exchange (ECDHE) → Both derive session keys → Finished (symmetric encryption begins).

Q15. What is the difference between public key and private key in asymmetric cryptography?

> Public key: shared openly, used to encrypt data or verify signatures. Private key: kept secret, used to decrypt data or create signatures. RSA: encrypt with recipient's public key, decrypt with private key. Signing: sign with private key, verify with public key.

Q16. What is NAT (Network Address Translation)?

> NAT allows multiple devices on a private network to share one public IP. Router maintains a translation table mapping private IP:port to public IP:port. Types: Static NAT, Dynamic NAT, PAT (Port Address Translation — most common, many-to-one).

Q17. What is the difference between a hub, switch, and router?

> Hub (Layer 1): broadcasts to all ports, no intelligence. Switch (Layer 2): forwards to specific port using MAC table, collision domains per port. Router (Layer 3): routes between networks using IP addresses, connects different networks/subnets.

Q18. What is SSL pinning?

> SSL pinning hardcodes the expected server certificate or public key in the client app. Rejects connections if the certificate doesn't match, even if it's signed by a trusted CA. Prevents MITM attacks. Used in mobile apps. Bypass via Frida/hooking.

Q19. What is a digital signature?

> A digital signature uses asymmetric crypto to authenticate data: sender hashes the message and encrypts the hash with their private key (signature). Receiver decrypts with sender's public key and compares to computed hash. Ensures integrity and non-repudiation.

Q20. What is DHCP and how does it work?

> DHCP (Dynamic Host Configuration Protocol) automatically assigns IP addresses. Process (DORA): Discover (client broadcasts) → Offer (server offers IP) → Request (client requests offered IP) → Acknowledge (server confirms). Assigns IP, subnet mask, gateway, DNS.

HARD (10 QUESTIONS)

Q21. How does AES key expansion (key schedule) work?

> AES key schedule derives round keys from the original key. For AES-128: original 16-byte key → expand to 176 bytes (11 round keys × 16 bytes). Each word is derived using SubBytes (S-box), RotWord, XOR with Rcon (round constant) for first word; XOR with previous round key for others.

Q22. What is the difference between AES-128, AES-192, and AES-256 security?

> AES-128: 10 rounds, key space 2^128 (~3.4×10^38). AES-192: 12 rounds, 2^192. AES-256: 14 rounds, 2^256. For classical computers, brute force is infeasible for all three. AES-256 is quantum-resistant (Grover's reduces to 2^128 effective). AES-128/256 are most commonly used.

Q23. What is BGP (Border Gateway Protocol)?

> BGP is the inter-domain routing protocol of the internet. It routes between Autonomous Systems (ASes). Uses path-vector algorithm. BGP hijacking (advertising false routes) is a known attack. BGP security: RPKI (Resource Public Key Infrastructure) validates route origins.

Q24. What is QUIC protocol?

> QUIC is a transport protocol developed by Google, basis of HTTP/3. Runs over UDP, implements reliability/congestion control in user space. Benefits over TCP+TLS: 0-RTT/1-RTT handshakes, no head-of-line blocking, connection migration (IP change without reconnect).

Q25. Explain how PKI (Public Key Infrastructure) works.

> PKI enables trusted public key distribution: Certificate Authorities (CAs) sign digital certificates binding a public key to an identity. Trust chain: Root CA → Intermediate CA → End-entity certificate. Browsers trust Root CAs in their trust store. CRLs/OCSP handle revocation.

Q26. What is a rainbow table attack and how is it mitigated?

> Rainbow tables precompute hash values for common passwords to crack hashes quickly. Mitigation: password salting — append a unique random value (salt) to each password before hashing. Salt is stored with hash. Makes precomputed tables useless since each password has a unique hash.

Q27. What is the difference between IDS and IPS?

> IDS (Intrusion Detection System): monitors network/system, detects suspicious activity, alerts but does NOT block. IPS (Intrusion Prevention System): detects + actively blocks/drops malicious traffic inline. IDS is passive; IPS is active. False positives in IPS can block legitimate traffic.

Q28. How does HTTPS prevent MITM attacks?

> TLS certificate authenticates the server's identity (signed by trusted CA). Client verifies: certificate is signed by trusted CA, not expired, hostname matches, not revoked (OCSP). Without the server's private key, attacker cannot decrypt traffic or impersonate server with a valid certificate.

Q29. What is Perfect Forward Secrecy (PFS)?

> PFS ensures that compromising the server's long-term private key doesn't decrypt past session traffic. Achieved using ephemeral Diffie-Hellman (DHE/ECDHE) key exchange — a new session key is generated per connection and discarded after use. Required for modern TLS deployments.

Q30. What is a side-channel attack on AES?

> Side-channel attacks exploit implementation leakage rather than cryptographic weakness: timing attacks (measuring encryption time reveals key bits), cache timing (FLUSH+RELOAD), power analysis (differential power analysis on embedded devices). Mitigated with constant-time implementations, hardware AES instructions (AES-NI).

End of Interview Preparation Guide Total: 12 Topics × 30 Questions = 360 Questions Good luck with your interview!